

Reviewer Pack — Minimal Number of Generators for Affine Group Bounds Monoton...

Entry 52242dd13976 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:02:15 UTC

Minimal Number of Generators for Affine Group Bounds Monotone Circuit Size

Entry ID: 52242dd13976 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 09:02:15 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Affine Groups) **Field B** (complexity object): Boolean Function Complexity (Monotone Circuit Size)

Statement:

[‘For every boolean function f with monotone circuit size $c(f)$, there exists an affine group G_f generated by at most $\alpha(c(f))$ elements, such that any element of the Boolean algebra with f as its characteristic polynomial is contained in a coset of G_f .’, ‘Here, $\alpha(n)$ is a monotonically increasing function with $\alpha(1) = 1$.’]

Rationale (proposer’s reasoning):

[‘Affine groups have been studied in algebraic combinatorics and their representation theory. They exhibit a rich structure that can potentially be used to simplify the computation of boolean functions represented as monotone circuits.’, ‘The conjecture aims to bridge this field with complexity theory by suggesting that the minimal number of generators for an affine group associated with a boolean function bounds its circuit size, which is a fundamental measure in computational complexity.’]

Taxonomy category: AffineGroupBoundsMonotoneCircuitSize (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c75ee637c1ee1d57

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the generated boolean functions f exhibit a correlation coefficient between $\alpha(c(f))$ and $c(f)$ greater than or equal to 0.8, with an aggregate mean absolute difference between $\alpha(c(f))$ and $c(f)$ across all seeds not exceeding 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine group" AND "monotone circuit size" - "characteristic polynomial" AND "affine group generators" - "Boolean algebra" AND "coset of affine group"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_characteristic_polynomial(f):
        n = len(f)
        A = [[0] * (n + 1) for _ in range(n + 1)]
        b = [0] * (n + 1)
        for i in range(n):
            for j in range(n):
                A[i][j] = f[2**i & 2**j]
            A[i][n] = -f[2**i]
            b[i] = f[2**i]
        return solve_linear_system(A, b)

    def solve_linear_system(A, b):
        n = len(b)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i + 1, n):
                factor = -A[j][i] / A[i][i]
```

```

        for k in range(i, n + 1):
            A[j][k] += factor * A[i][k]
            b[j] += factor * b[i]
x = [0] * n
for i in range(n - 1, -1, -1):
    x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
return x

def count_nonzero_entries(v):
    return sum(1 for entry in v if entry != 0)

def generate_affine_group(f):
    n = len(f)
    G = []
    for i in range(2**n):
        g = [f[i ^ j] for j in range(n)]
        G.append(g)
    return G

def is_coset(G, f):
    n = len(f)
    for g in G:
        if all((g[i] + f[i]) % 2 == 0 for i in range(n)):
            return True
    return False

def alpha(c):
    return c # Placeholder for actual function  $\alpha(n)$ 

results = []
n_values = [5, 10, 15, 20, 30, 40]
for n in n_values:
    f = generate_boolean_function(n)
    char_poly = compute_characteristic_polynomial(f)
    G = generate_affine_group(char_poly)
    num_generators = count_nonzero_entries(G[0])
    c_f = len(f) - 1
    results.append({
        "n": n,
        "num_generators": num_generators,
        "circuit_size": c_f,
        "alpha_c_f": alpha(c_f),
        "is_coset": is_coset(G, f)
    })

metric_value = sum(result["alpha_c_f"] - result["circuit_size"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(abs(result["alpha_c_f"] - result["circuit_size"]) <= 3 for result in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "mean_absolute_difference",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

```

```

}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_142002de.py", line 110, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_142002de.py", line 79, in run_trial
    char_poly = compute_characteristic_polynomial(f)
                ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_142002de.py", line 30, in compute_characteristic_polynomial
    A[i][j] = f[2*i & 2*j]
              ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means we cannot verify the pre-registered support condition for the conjecture. | next: Re-run the test to ensure it completes successfully and produces the required data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11421
2	preregistration	ollama_remote	glm4:latest	0	0	5749
3	novelty	ollama_remote	glm4:latest	0	0	4780
4	novelty	ollama_remote	glm4:latest	0	0	5334
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13170
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13905
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10278
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13336
9	judge	ollama_remote	glm4:latest	0	0	7992

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 85965 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/52242dd13976.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/52242dd13976.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/52242dd13976.tar.gz* (if generated)